

作业 HW6-排序 实验报告

姓名：毛星博 学号：2351520 日期：2025 年 4 月 26 日

1. 涉及数据结构和相关背景

排序算法是计算机科学中最基本也是最重要的算法之一，它们被广泛应用于数据处理、搜索优化和计算机图形学等领域。

1.1 插入排序类

插入排序类算法的基本思想是将一个新元素插入到已排序好的序列中，形成一个新的有序序列。

1.1.1 直接插入排序

基本思路：从第二个元素开始，将其与前面已排序的元素比较，找到合适位置插入。

实现方式：通过不断将元素与前面的元素比较并交换位置，直到找到正确的位置。

时间复杂度：

- 最好情况： $O(n)$ （数组已经有序）
- 最坏情况： $O(n^2)$ （数组逆序）
- 平均情况： $O(n^2)$

空间复杂度： $O(1)$ ，原地排序算法

稳定性：稳定。在插入过程中，相等元素的相对位置不会改变。

1.2 折半插入排序

基本思路：利用二分查找来加速查找插入位置的过程。

实现方式：对于每个新元素，使用二分查找在已排序部分找到插入位置，然后移动元素并插入。

时间复杂度：

- 查找操作： $O(\log n)$
- 移动操作： $O(n)$
- 总体时间复杂度： $O(n^2)$

空间复杂度： $O(1)$

稳定性：稳定。二分查找只影响查找插入位置的效率，不影响元素的相对顺序。

1.3 表插入排序

基本思路：使用链表结构进行插入排序，避免数组移动元素的开销。

实现方式：维护一个有序链表，每次将新元素插入到链表的适当位置。

时间复杂度： $O(n^2)$

空间复杂度： $O(n)$ （需要额外的链表结构）

稳定性：稳定。可以保证相等元素的相对位置不变。

1.4 希尔排序

基本思路：将序列分成若干子序列，对每个子序列进行直接插入排序，逐步缩小增量直到为 1。

实现方式：按照增量序列（如 Knuth 序列）将数组分组，对每组进行插入排序，然后减小增量再排序。

时间复杂度：取决于增量序列，一般为 $O(n^{1.3})$ 到 $O(n^2)$ 之间

空间复杂度： $O(1)$

稳定性：不稳定。由于希尔排序是跳跃式的插入排序，相同元素可能会被分到不同的子序列中，导致相对位置改变。

2. 交换排序类

交换排序类算法的基本思想是通过交换元素位置来达到排序的目的。

2.1 起泡排序（冒泡排序）

基本思路：重复遍历序列，比较相邻元素并交换不符合顺序的元素对。

实现方式：每次遍历将最大（或最小）元素“冒泡”到序列的一端。

时间复杂度：

- 最好情况： $O(n^2)$
- 最坏情况： $O(n^2)$
- 平均情况： $O(n^2)$

空间复杂度： $O(1)$

稳定性：稳定。只有当相邻元素大小关系不满足要求时才交换，相等元素不会交换，因此保持原有顺序。

2.2 快速排序

基本思路：选择一个基准元素，将序列分为两部分，一部分小于基准，一部分大于基准，然后递归排序两个子序列。

实现方式：通过分区（partition）操作将数组分为两部分，然后递归处理。

时间复杂度：

- 最好情况： $O(n \log n)$
- 最坏情况： $O(n^2)$ （当数组已排序时）
- 平均情况： $O(n \log n)$

空间复杂度： $O(\log n)$ （递归调用栈）

稳定性：不稳定。在分区过程中，元素的相对位置可能发生改变，特别是与基准元素相等的元素。

3. 选择排序类

选择排序类算法的基本思想是每次从未排序部分选择最小（或最大）的元素放到已排序部分的末尾。

3.1 简单选择排序

基本思路：每次从未排序部分找出最小元素，放到已排序部分的末尾。

实现方式：通过 $n-1$ 次遍历，每次找出最小元素并交换到前面。

时间复杂度： $O(n^2)$ （无论输入如何）

空间复杂度： $O(1)$

稳定性：不稳定。在交换过程中，相等元素的相对位置可能发生改变。例如，当最小元素与前面相等的元素交换位置时。

3.2 树形选择排序（锦标赛排序）

基本思路：使用完全二叉树结构进行比较，找出最小元素。

实现方式：构建一个胜者树，每次选出最小值后重新调整树结构。

时间复杂度： $O(n \log n)$

空间复杂度： $O(n)$

稳定性：稳定。可以通过适当的实现保证相等元素的相对位置不变。

3.3 堆排序

基本思路：利用堆数据结构（通常是最大堆）进行排序。

实现方式：先构建最大堆，然后每次取出堆顶元素（最大值）放到数组末尾，并重新调整堆。

时间复杂度： $O(n \log n)$

空间复杂度： $O(1)$

稳定性：不稳定。在堆调整过程中，相等元素的相对位置可能发生改变。

4. 归并排序

基本思路：将序列分成两部分，分别排序后再合并。

实现方式：递归地将数组分为两半，排序后再合并两个有序数组。

时间复杂度： $O(n \log n)$ （各种情况下都是）

空间复杂度： $O(n)$ （需要额外空间来合并数组）

稳定性：稳定。在合并过程中，可以保证相等元素的相对位置不变。当两个元素相等时，可以先选择前一个子数组中的元素，保证稳定性。

5. 基数排序

基本思路：按照元素的位数进行多次分配和收集。

实现方式：从最低位（或最高位）开始，对每一位进行桶排序，然后合并

时间复杂度： $O(d(n+r))$ ，其中 d 是位数， r 是基数（如十进制 $r=10$ ）

空间复杂度： $O(n+r)$

稳定性：稳定。在每一轮的分配和收集过程中，相等元素的相对位置不会改变。

2. 实验内容

2.1 求逆序对

2.1.1 问题描述

对于一个长度为 N 的整数序列 A ，满足 $i < j$ 且 $A_i > A_j$ 的数对 (i,j) 称为整数序列 A 的一个逆序。请求出整数序列 A 的所有逆序对个数。

2.1.2 基本要求

输入：

第一行两个整数 n 和 k ，分别表示 $nums$ 的元素个数 n ，和向右轮转 k 个位置；

第二行包括 n 个整数，为顺序表 $nums$ 中的元素

输出：

轮转后的顺序表中的元素

2.1.3 数据结构设计

```
//合并子序列时临时存放排好序的元素，以避免在原数组上直接操作造成的干扰
vector<int> temp(n);
//存储输入的整数序列
vector<int> arr(n);
```

2.1.4 功能说明（函数、类）

```
// 归并过程中统计逆序对
//返回值：本次合并过程中发现的跨越两个子序列的逆序对数量。
long long merge(vector<int>& arr, vector<int>& temp, int left, int mid, int right) {
    int i = left;    // 左半部分起始位置
```

```

    int j = mid + 1; // 右半部分起始位置
    int k = left;    // 临时数组起始位置
    long long inversions = 0; // 逆序对计数
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            // 没有逆序对
            temp[k++] = arr[i++];
        } else {
            // 形成逆序对，左边元素大于右边元素
            // 因为左半部分已排序，所以从 i 到 mid 的所有元素都会与 arr[j] 形成逆序对
            inversions += (mid - i + 1);
            temp[k++] = arr[j++];
        }
    }
    // 处理剩余元素
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];
    // 将临时数组中已排序的部分复制回原数组
    for (i = left; i <= right; i++) arr[i] = temp[i];
    return inversions;
}

// 归并排序统计逆序对
// 返回值指定范围 [left, right] 内的总逆序对数量。
long long mergeSort(vector<int>& arr, vector<int>& temp, int left, int right) {
    long long inversions = 0;
    if (left < right) {
        int mid = left + (right - left) / 2;
        // 计算左半部分的逆序对
        inversions += mergeSort(arr, temp, left, mid);
        // 计算右半部分的逆序对
        inversions += mergeSort(arr, temp, mid + 1, right);
        // 计算跨越左右两部分的逆序对
        inversions += merge(arr, temp, left, mid, right);
    }
    return inversions;
}

```

2.1.5 调试分析（遇到的问题和解决方法）

在解决求逆序对数问题时，最初的想法是使用双重循环进行暴力枚举。即遍历所有可能的数对 (i, j) ，其中 $i < j$ ，然后判断是否满足 $A[i] > A[j]$ 。如果满足，则逆序对计数加一。这种暴力枚举方法的时间复杂度为 $O(N^2)$ ，计算量过大，会导致程序运行超时，无法通过所有测试用例。经过分析和学习，发现归并排序算法可以在排序的同时高效地统计逆序对。在归并排序的合并

(merge) 阶段, 当我们需要将右侧子数组的元素 `arr[j]` 放入临时数组, 且此时左侧子数组的指针 `i` 指向的元素 `arr[i]` 大于 `arr[j]` 时, 就意味着左侧子数组中从 `arr[i]` 到 `arr[mid]` 的所有元素都与 `arr[j]` 构成了逆序对。利用这一特性, 可以在 $O(N \log N)$ 的时间复杂度内完成排序和逆序对统计。因此, 改用基于归并排序的方法来实现, 成功解决了时间复杂度过高的问题。

2.1.6 总结和体会

1. 算法设计中时间复杂度的重要性, 选择合适的算法对程序性能至关重要。
2. 归并排序不仅是一种高效的排序算法, 其分治思想和合并过程还可以被巧妙地应用于解决其他问题, 如计算逆序对。
3. 理解算法的内在原理有助于我们发现其潜在的应用价值。认识到 `merge` 操作中比较大小的步骤天然地揭示了逆序对的存在, 是解决此问题的关键。
4. 编写代码不仅要实现功能, 还要考虑效率和资源消耗, 针对具体问题选择最优的数据结构和算法是提升程序质量的有效途径。

2.2 最大数

2.2.1 问题描述

给定一组非负整数 `nums`, 重新排列每个数的顺序 (每个数不可拆分) 使之组成一个最大的整数。
注意: 输出结果可能非常大, 所以你需要返回一个字符串而不是整数。

2.2.2 基本要求

输入描述

输入包含两行 第一行包含一个整数 `n`, 表示组数 `nums` 的长度

第二行包含 `n` 个整数 `nums[i]`

输出描述

输出包含一行, 为重新排列后得到的数字

2.2.3 数据结构设计

```
vector<int> nums;
```

2.2.4 功能说明 (函数、类)

```
class Solution{
private:
    //这是一个自定义的比较函数, 用于确定两个数字字符串 a 和 b 的拼接顺序。
    static bool compare(const string& a, const string& b){return a+b>b+a;}
    //冒泡排序
    void BubbleSort(vector<string>& strs){
        int n=strs.size();
        for(int i=0;i<n-1;i++){
            for(int j=0;j<n-i-1;j++){
                if(!compare(strs[j],strs[j+1])){
                    swap(strs[j],strs[j+1]);
                }
            }
        }
    }
}
```

```

    }
}

public:
    string largestNumber(vector<int>& nums){
        vector<string> strs;
        for(int num:nums){
            strs.push_back(to_string(num)); //将整数转换为字符串并添加到字符串数组中
        }
        BubbleSort(strs); //对字符串数组进行排序
        if(strs[0]=="0") return "0";
        string res;
        for(string& str:strs){
            res+=str;
        }
        return res;
    }
};

```

2.2.5 调试分析（遇到的问题解决方法）

最初的想法是直接对数字进行某种数值上的比较，但这很快会发现行不通。例如，对于 $[3, 30, 34, 5, 9]$ ，简单的数值排序 $(3, 5, 9, 30, 34)$ 或者逆序 $(34, 30, 9, 5, 3)$ 都无法直接得到最大组合 "9534330"。

通过分析发现，决定两个数字 a 和 b 先后顺序的正确方法是比较两种拼接方式 $a + b$ 和 $b + a$ 的字典序大小。如果 $a + b > b + a$ ，则 a 应该排在 b 前面。基于这个发现，我定义了一个自定义比较函数 `compare`。然后，将所有输入数字转换为字符串，并使用这个自定义比较函数对字符串进行排序（代码中使用了冒泡排序，也可以使用 `std::sort`）。排序后，将字符串按顺序拼接起来即可得到最大的整数。

2.2.6 总结和体会

1. 排序问题的关键有时并非数值大小本身，而是根据特定目标定义的相对顺序。
2. 将数字转换为字符串处理是解决这类问题的有效技巧，可以方便地实现拼接比较。

2.3 排序

2.3.1 问题描述

排序算法分为简单排序（时间复杂度为 $O(n^2)$ ）和高效排序（时间复杂度为 $O(n \log n)$ ）。

本题给定 N 个整数，要求输出从小到大排序后的结果。请用不同的排序算法测试，注意有些算法无法拿到满分，由此同学们可以猜测一下 10 个测试用例的数据特征。

请同学们自己随机生成不同规模的数据（例如 10, 100, 1K, 10K, 100K, 1M, 10K 正序, 10K 逆序），用不同的排序算法（快速排序，归并排序，堆排序，选择排序，冒泡排序，直接插入排序，希尔排序）分别对这些数据进行测试，输出运行时间，将结果写在实验报告中，并总结各种排序算法的特点、时间复杂度、空间复杂度，以及是否是稳定排序和部分排序。

2.3.2 基本要求

输入

第 1 行一个正整数 n ，表示元素个数

第 2 行 n 个整数，用空格分割

输出

从小到大排序后的结果，以空格分割。

2.3.3 排序算法设计

```
// --- 快速排序 ---
int partition(std::vector<int> &nums, int l, int r) {
    int pivot = nums[l];
    int i = l, j = r;
    while (i < j) {
        while (i < j && nums[j] >= pivot) j--; // 从右向左找第一个小于 pivot 的数
        if (i < j) nums[i++] = nums[j];      // 放入左边
        while (i < j && nums[i] <= pivot) i++; // 从左向右找第一个大于 pivot 的数
        if (i < j) nums[j--] = nums[i];      // 放入右边
    }
    nums[i] = pivot; // 基准元素归位
    return i;       // 返回基准元素最终位置
}

void quickSortRecursive(std::vector<int> &nums, int l, int r) {
    if (l >= r) return; // 递归终止条件
    int pivot = partition(nums, l, r); // 划分
    quickSortRecursive(nums, l, pivot - 1); // 递归排序左半部分
    quickSortRecursive(nums, pivot + 1, r); // 递归排序右半部分
}

// --- 归并排序 ---
void mergeSortRecursive(std::vector<int> &nums, int l, int r) {
    if (l >= r) return; // 递归终止条件
    int mid = l + (r - l) / 2; // 防止溢出
    mergeSortRecursive(nums, l, mid); // 递归排序左半部分
    mergeSortRecursive(nums, mid + 1, r); // 递归排序右半部分

    // 合并两个有序子数组
    int i = l, j = mid + 1, k = 0;
    std::vector<int> tmp(r - l + 1); // 临时数组
    while (i <= mid && j <= r) {
        if (nums[i] <= nums[j]) tmp[k++] = nums[i++];
        else tmp[k++] = nums[j++];
    }
    // 处理剩余元素
    while (i <= mid) tmp[k++] = nums[i++];
    while (j <= r) tmp[k++] = nums[j++];
    // 将临时数组复制回原数组
```

```

        for (int idx = 0; idx < tmp.size(); ++idx) nums[l + idx] = tmp[idx];
    }

    // --- 堆排序 ---
    // 调整堆，使其满足最大堆性质
    void heapify(std::vector<int>& nums, int n, int i) {
        int largest = i;    // 初始化最大值为根节点
        int l = 2 * i + 1;  // 左子节点
        int r = 2 * i + 2;  // 右子节点
        // 如果左子节点大于根节点
        if (l < n && nums[l] > nums[largest])
            largest = l;
        // 如果右子节点大于当前最大值
        if (r < n && nums[r] > nums[largest])
            largest = r;
        // 如果最大值不是根节点
        if (largest != i) {
            std::swap(nums[i], nums[largest]); // 交换
            heapify(nums, n, largest);         // 递归调整受影响的子树
        }
    }

    void heapSort(std::vector<int>& nums) {
        int n = nums.size();
        // 构建最大堆（从最后一个非叶子节点开始调整）
        for (int i = n / 2 - 1; i >= 0; i--)
            heapify(nums, n, i);
        // 逐个提取元素
        for (int i = n - 1; i > 0; i--) {
            std::swap(nums[0], nums[i]); // 将当前最大元素（堆顶）移到末尾
            heapify(nums, i, 0);         // 重新调整剩余元素的堆结构
        }
    }

    // --- 选择排序 ---
    void selectionSort(std::vector<int>& nums) {
        int n = nums.size();
        for (int i = 0; i < n - 1; i++) {
            int min_idx = i; // 记录最小元素的索引
            // 在未排序部分找到最小元素
            for (int j = i + 1; j < n; j++)
                if (nums[j] < nums[min_idx])
                    min_idx = j;
            // 如果最小元素不是当前位置元素，则交换
            if (min_idx != i)
                std::swap(nums[i], nums[min_idx]);
        }
    }

```

```

}

// --- 冒泡排序 ---
void bubbleSort(std::vector<int>& nums) {
    int n = nums.size();
    bool swapped; // 优化: 标记是否发生交换
    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        // 比较相邻元素
        for (int j = 0; j < n - i - 1; j++) {
            if (nums[j] > nums[j + 1]) {
                std::swap(nums[j], nums[j + 1]);
                swapped = true;
            }
        }
        // 如果一轮比较中没有发生交换, 说明数组已经有序
        if (!swapped) break;
    }
}

// --- 插入排序 ---
void insertionSort(std::vector<int>& nums) {
    int n = nums.size();
    for (int i = 1; i < n; i++) {
        int key = nums[i]; // 当前待插入元素
        int j = i - 1;
        // 将大于 key 的元素向后移动
        while (j >= 0 && nums[j] > key) {
            nums[j + 1] = nums[j];
            j = j - 1;
        }
        nums[j + 1] = key; // 插入 key 到正确位置
    }
}

// --- 希尔排序 ---
void shellSort(std::vector<int>& nums) {
    int n = nums.size();
    // 使用 Knuth 增量序列: ..., 121, 40, 13, 4, 1
    int gap = 1;
    while (gap < n / 3) {
        gap = 3 * gap + 1;
    }
    // 逐步减小增量
    for (; gap > 0; gap /= 3) {
        // 对每个间隔为 gap 的子序列进行插入排序
    }
}

```

```
for (int i = gap; i < n; i += 1) {
    int temp = nums[i];
    int j;
    for (j = i; j >= gap && nums[j - gap] > temp; j -= gap)
        nums[j] = nums[j - gap];
    nums[j] = temp;
}
```

2.3.4 测试函数设计

```
// 定义待测试算法
std::map<std::string, std::function<std::vector<int>(Solution&, std::vector<int>&)>>
algorithms;

algorithms["快速排序"] = &Solution::runQuickSort;
algorithms["归并排序"] = &Solution::runMergeSort;
algorithms["堆排序"] = &Solution::runHeapSort;
algorithms["希尔排序"] = &Solution::runShellSort;
algorithms["插入排序"] = &Solution::runInsertionSort; //  $O(n^2)$ 
algorithms["选择排序"] = &Solution::runSelectionSort; //  $O(n^2)$ 
algorithms["冒泡排序"] = &Solution::runBubbleSort; //  $O(n^2)$ 

// 定义数据规模
std::vector<int> sizes = {10, 100, 1000, 10000, 100000, 1000000};
int specificSize = 10000; // 特定测试规模（用于有序和逆序数据）
int skipThreshold = 100000; //  $O(n^2)$  算法跳过测试的阈值

// 输出表头
std::cout << std::left << std::setw(15) << "算法名称"
    << std::setw(12) << "数据规模"
    << std::setw(15) << "数据类型"
    << std::setw(15) << "运行时间 (ms)" << std::endl;

std::cout << std::string(60, '-') << std::endl;

// 遍历每种算法
for (const auto& pair : algorithms) {
    const std::string& name = pair.first;
    auto sortFunc = pair.second;
    bool isN2 = (name == "插入排序" || name == "选择排序" || name == "冒泡排序");
    // 测试不同规模的随机数据
    for (int size : sizes) {
        // 如果是  $O(n^2)$  算法且数据规模过大，则跳过
        if (isN2 && size >= skipThreshold) {
            std::cout << std::left << std::setw(15) << name
                << std::setw(12) << size
                << std::setw(15) << "随机"
```

```

        << std::setw(15) << "跳过 ( $O(N^2)$ )" << std::endl;
        continue;
    }
    std::vector<int> data = generateRandomData(size);
    std::vector<int> data_copy = data; // 对副本进行排序, 保留原始数据
    // 计时开始
    auto start = std::chrono::high_resolution_clock::now();
    sortFunc(s, data_copy); // 执行排序
    auto end = std::chrono::high_resolution_clock::now();
    // 计算耗时
    std::chrono::duration<double, std::milli> duration = end - start;
    // 输出结果
    std::cout << std::left << std::setw(15) << name
        << std::setw(12) << size
        << std::setw(15) << "随机"
        << std::fixed << std::setprecision(3) << std::setw(15) << duration.count()
<< std::endl;
    }
    // 测试 10K 有序数据
    {
        int size = specificSize;
        std::vector<int> data = generateSortedData(size);
        std::vector<int> data_copy = data;
        auto start = std::chrono::high_resolution_clock::now();
        sortFunc(s, data_copy);
        auto end = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double, std::milli> duration = end - start;
        std::cout << std::left << std::setw(15) << name
            << std::setw(12) << size
            << std::setw(15) << "有序"
            << std::fixed << std::setprecision(3) << std::setw(15) << duration.count()
<< std::endl;
    }
    // 测试 10K 逆序数据
    {
        int size = specificSize;
        std::vector<int> data = generateReverseSortedData(size);
        std::vector<int> data_copy = data;
        auto start = std::chrono::high_resolution_clock::now();
        sortFunc(s, data_copy);
        auto end = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double, std::milli> duration = end - start;
        std::cout << std::left << std::setw(15) << name
            << std::setw(12) << size

```

```

        << std::setw(15) << "逆序"
        << std::fixed << std::setprecision(3) << std::setw(15) << duration.count()
<< std::endl;
    }
    std::cout << std::string(60, '-') << std::endl; // 分隔线
}

```

2.3.5 测试结果

算法名称	数据规模	数据类型	运行时间 (ms)

插入排序	10	随机	0.000
插入排序	100	随机	0.000
插入排序	1000	随机	0.000
插入排序	10000	随机	83.201
插入排序	100000	随机	跳过 ($O(N^2)$)
插入排序	1000000	随机	跳过 ($O(N^2)$)
插入排序	10000	有序	0.000
插入排序	10000	逆序	173.380

堆排序	10	随机	0.000
堆排序	100	随机	0.000
堆排序	1000	随机	0.000
堆排序	10000	随机	2.660
堆排序	100000	随机	28.404
堆排序	1000000	随机	327.338
堆排序	10000	有序	0.000
堆排序	10000	逆序	0.000

归并排序	10	随机	0.000
归并排序	100	随机	0.000
归并排序	1000	随机	0.000
归并排序	10000	随机	9.445
归并排序	100000	随机	30.074
归并排序	1000000	随机	333.819
归并排序	10000	有序	0.000
归并排序	10000	逆序	8.735

快速排序	10	随机	0.000
快速排序	100	随机	0.000
快速排序	1000	随机	0.000
快速排序	10000	随机	0.000
快速排序	100000	随机	15.614
快速排序	1000000	随机	113.564
快速排序	10000	有序	61.925

快速排序	10000	逆序	61.096

冒泡排序	10	随机	0.000
冒泡排序	100	随机	0.000
冒泡排序	1000	随机	6.198
冒泡排序	10000	随机	309.517
冒泡排序	100000	随机	跳过 ($O(N^2)$)
冒泡排序	1000000	随机	跳过 ($O(N^2)$)
冒泡排序	10000	有序	0.000
冒泡排序	10000	逆序	334.151

希尔排序	10	随机	0.000
希尔排序	100	随机	0.000
希尔排序	1000	随机	0.000
希尔排序	10000	随机	0.000
希尔排序	100000	随机	30.019
希尔排序	1000000	随机	346.130
希尔排序	10000	有序	0.000
希尔排序	10000	逆序	0.000

选择排序	10	随机	0.000
选择排序	100	随机	0.000
选择排序	1000	随机	0.000
选择排序	10000	随机	103.175
选择排序	100000	随机	跳过 ($O(N^2)$)
选择排序	1000000	随机	跳过 ($O(N^2)$)
选择排序	10000	有序	102.163
选择排序	10000	逆序	106.648

2.3.6 总结和体会（更详细的内容在第一部分已经介绍）

快速排序 (Quick Sort):

平均时间复杂度: $O(N \log N)$

最坏时间复杂度: $O(N^2)$ (当输入数据已排序或接近排序)

最好时间复杂度: $O(N \log N)$ (当每次划分都接近均分时)

空间复杂度: $O(\log N)$ (递归调用栈的深度)

稳定性: 不稳定

是否部分排序: 不是

归并排序 (Merge Sort):

平均时间复杂度: $O(N \log N)$

最坏时间复杂度: $O(N \log N)$

最好时间复杂度: $O(N \log N)$

空间复杂度: $O(N)$ (需要额外的临时数组来合并)

稳定性: 稳定

是否部分排序: 不是

堆排序 (Heap Sort):

平均时间复杂度: $O(N \log N)$

最坏时间复杂度: $O(N \log N)$

最好时间复杂度: $O(N \log N)$

空间复杂度: $O(1)$ (原地排序)

稳定性: 不稳定

是否部分排序: 是

希尔排序 (Shell Sort):

平均时间复杂度: 取决于增量序列, 通常优于 $O(N^2)$, 常见如 $O(N^{1.3})$ 或 $O(N \log^2 N)$

最坏时间复杂度: $O(N^2)$ (使用不佳的增量序列, 如 1, 2, 4, ...)

最好时间复杂度: $O(N \log N)$ (对于某些特定的增量序列)

空间复杂度: $O(1)$ (原地排序)

稳定性: 不稳定

是否部分排序: 不是

插入排序 (Insertion Sort):

平均时间复杂度: $O(N^2)$

最坏时间复杂度: $O(N^2)$ (当输入数据逆序时)

最好时间复杂度: $O(N)$ (当输入数据已排序时)

空间复杂度: $O(1)$ (原地排序)

稳定性: 稳定

是否部分排序: 不是

选择排序 (Selection Sort):

平均时间复杂度: $O(N^2)$

最坏时间复杂度: $O(N^2)$

最好时间复杂度: $O(N^2)$ (比较次数不变)

空间复杂度: $O(1)$ (原地排序)

稳定性: 不稳定 (选择最小值交换时可能改变相等元素的相对顺序)

是否部分排序: 是

冒泡排序 (Bubble Sort):

平均时间复杂度: $O(N^2)$

最坏时间复杂度: $O(N^2)$

最好时间复杂度: $O(N^2)$

空间复杂度: $O(1)$ (原地排序)

稳定性: 稳定

是否部分排序: 不是

2.4 序列

2.4.1 问题描述

给定 m 个数字序列, 每个序列包含 n 个非负整数。我们从每一个序列中选取一个数字组成一个新的序列, 显然一共可以构造出 n^m 个新序列。接下来我们对每一个新的序列中的数字进行求和, 一共会得到 n^m 个和, 请找出最小的 n 个和

2.4.2 基本要求

输入格式:

输入的第一行是一个整数 T ，表示测试用例的数量，接下来是 T 个测试用例的输入
每个测试用例输入的第一行是两个正整数 m ($0 < m \leq 100$) 和 n ($0 < n \leq 2000$),
然后有 m 行，每行有 n 个数，数字之间用空格分开，表示这 m 个序列
序列中的数字不会大于 10000

输出格式:

对每组测试用例，输出一行用空格隔开的数，表示最小的 n 个和

2.4.3 数据结构设计

```
//sum 通过不断更新累加保持个数为 n 个 即输入 i 行之后，前 i 行各出一个元素的前 n 小和数组
//当输入完 m 行后，sum 即为前 m 行各出一个元素的前 n 小和数组，即为最终答案
//有点局部最优解的思想？
int sum[MAXN];
//new_line 代表新输入的一行
int new_line[MAXN];
```

2.4.4 功能说明（函数、类）

```
/*通过两层循环遍历 sum 数组和 new_line 数组的所有元素对的和。
利用 std::priority_queue<int>（大顶堆）来维护当前找到的 n 个最小的和。
对于每个新计算出的和 new_sum，如果堆的大小小于 n，直接将其推入堆；
如果堆已满，则仅当 new_sum 小于堆顶元素（当前最大的 n 个和中的最大值）时，才弹出堆顶元素并将 new_sum 推入。最后，将堆中的 n 个元素（即新的前 n 小和）按降序（因为是大顶堆）取出并反向存入 sum 数组，使其保持升序。*/
void update_sum(int* sum, int* new_line, int n)
{
    priority_queue<int> q; // 大根堆
    for (int j = 0; j < n; ++j){
        for (int i = 0; i < n; ++i){
            int new_sum = sum[i] + new_line[j];
            if (q.size() < n)
                q.push(new_sum);
            else if (new_sum < q.top()){
                q.pop(); // 替换最大元素
                q.push(new_sum);
            }
            else break; //因为两个序列均有序 若当前已经超过，则之后均不会再更新
        }
    }
    for (int i = n - 1; i >= 0; i--){
        sum[i] = q.top();
        q.pop();
    }
}
```

```
}
```

2.4.5 调试分析（遇到的问题解决方法）

最直接的想法是枚举所有 nm 种组合，计算它们的和，然后排序取前 n 个。但当 n 和 m 较大时， nm 会变得极其巨大，导致时间和空间复杂度都无法接受。即使不枚举所有组合，如果尝试存储中间过程产生的大量候选和，也可能超出内存限制。

所以采用了一种逐步合并的方法，并结合优先队列（大顶堆）进行优化。先将第一行排序作为初始的 n 个最小和。然后逐行读入新的序列，排序后，将其与当前已有的 n 个最小和进行“合并”，生成新的 n 个最小和。这种方法只需要存储当前行的序列（`new_line`）和当前的前 n 小和（`sum`），以及一个大小为 n 的优先队列，避免了存储大量中间状态。

2.4.6 总结和体会

1. 分治和逐步合并的思想可以用来分解复杂问题。本题通过逐行处理，将 m 个序列的问题分解为 $m-1$ 次“合并两个结果集”的过程。
2. 利用数据的有序性进行剪枝是常见的优化手段，可以显著减少计算量。
3. 需要关注算法的空间复杂度，选择合适的策略避免内存溢出。

2.5 三数之和

2.5.1 问题描述

给你一个整数数组 `nums`，判断是否存在三元组 $[nums[i], nums[j], nums[k]]$ 满足 $i \neq j$ 、 $i \neq k$ 且 $j \neq k$ ，同时还满足 $nums[i] + nums[j] + nums[k] == 0$ 。请你返回所有和为 0 且不重复的三元组，每个三元组占一行。

2.5.2 基本要求

输入描述

2 行

第 1 行一个整数，表示数组元素个数；

第 2 行输入一组整数，中间以空格分隔。

输出描述

输出所有和为 0 的三元组，每个三元组一行，中间以空格分隔。

对于每一个三元组，你需要按从小到大的顺序依次返回三个元素；

对于所有三元组，你需要按三元组中最小元素从小到大的顺序依次打印每一组三元组。

注意：答案中不可以包含重复的三元组。

2.5.3 数据结构设计

```
vector<vector<int>> res=threeSum(nums);
```

2.5.4 功能说明（函数、类）

```
vector<vector<int>> threeSum(vector<int>& nums)
{
    vector<vector<int>> res;
    QuickSort(nums,0,nums.size()-1); // 对数组进行排序
```

```

for(int i=0;i<nums.size()-2;i++){
    if(nums[i]>0) break; // 如果当前数字大于 0，则三数之和一定大于 0
    if(i>0 && nums[i]==nums[i-1]) continue; // 跳过重复的元素
    int left=i+1,right=nums.size()-1; // 双指针
    while(left<right){
        int sum=nums[i]+nums[left]+nums[right]; // 计算三数之和
        if(sum==0){
            res.push_back({nums[i],nums[left],nums[right]});
            while(left<right && nums[left]==nums[left+1]) left++; // 跳过重复元素
            while(left<right && nums[right]==nums[right-1]) right--;
            left++;
            right--;
        }
        else if(sum<0) left++; // 如果和小于 0，左指针右移
        else right--; // 如果和大于 0，右指针左移
    }
}
return res;
}

```

2.5.5 调试分析（遇到的问题和解决方法）

如何确保最终结果中不包含重复的三元组？

当双指针找到一个解（`nums[i], nums[left], nums[right]`）时，在移动 `left` 和 `right` 指针之前，要持续移动它们，跳过所有与 `nums[left]` 和 `nums[right]` 重复的元素，以确保不会因为数组中的重复值而产生重复的三元组结果。

2.5.6 总结和体会

1. 双指针技巧：双指针是在有序数组（或部分有序结构）中查找满足特定条件的元素的一种高效的技巧。
3. 去重：在处理包含重复元素的输入时，去重是一个必须仔细考虑的问题。本题中通过在固定第一个数和找到解后两个环节进行判断和指针移动，有效地避免了结果重复。

3. 实验总结

本次系列实验围绕排序及其相关应用展开，涵盖了从基础排序算法性能比较到利用排序思想解决特定问题的多个方面。通过亲手实现和测试不同的算法，我对数据结构与算法的设计、选择、优化及其对程序性能的深刻影响有了更全面的认识。

求逆序对清晰地展示了算法选择对效率的巨大影响。暴力枚举法虽然直观，但 $O(N^2)$ 的复杂度使其在稍大数据规模下就不可行。而归并排序在 $O(N \log N)$ 时间内不仅完成了排序，还巧妙地利用合并过程统计了逆序对。这让我认识到，深入理解经典算法的内在过程，可以帮助我们发现解决其他相关问题的高效途径。

最大数的核心在于理解排序的本质是建立元素间的“序关系”。这个“序”不一定是数值大小，而是由问题目标决定的。通过定义 $a+b > b+a$ 这样的自定义比较规则，并将数字转为字符串处理，成功解决了看似棘手的最大数拼接问题。这让我学到，面对非常规排序需求时，要灵活运用自定义比较函数和数据类型转换。

排序性能测试通过对多种经典排序算法（快速排序、归并排序、堆排序、希尔排序、插入排序、选择排序、冒泡排序）在不同数据规模和数据特性（随机、有序、逆序）下的性能进行测试，我直观地感受到了 $O(N \log N)$ 算法相对于 $O(N^2)$ 算法在效率上的压倒性优势（刚开始没有跳过大规模数据，电脑风扇都要转冒烟了）。同时也观察到，某些算法（如快速排序）在特定输入下（如已排序或逆序）可能性能退化，而另一些（如插入排序、冒泡排序优化版）在数据近乎有序时表现优异。这强调了在实际应用中，不仅要考虑平均时间复杂度，还要结合数据特征和最坏情况来选择或优化排序算法。

序列处理的是一个组合数量可能爆炸性增长的问题 (N^M)。直接枚举不可行，这迫使我寻找更精巧的策略。通过学习和实现，我掌握了使用优先队列（堆）来高效解决问题的经典方法。逐行合并的思想，结合大顶堆动态维护当前最小的 n 个和，并利用有序性剪枝，有效地控制了时间和空间复杂度，避免了内存溢出。

三数之和通过首先对数组进行排序，使得后续利用双指针技术从 $O(N^3)$ 优化到 $O(N^2)$ 成为可能。双指针法在有序数组中查找满足特定和的元素对非常高效。同时，此题也强调了去重逻辑，需要分别在固定第一个元素和找到解后进行判断和移动指针，以确保结果的唯一性。

通过这一系列实验，我深刻认识到，算法和数据结构是计算机科学的核心。选择正确的算法和数据结构对于程序的性能有着决定性的影响。没有“万能”的算法，每种算法都有其优势、劣势和适用场景。理解不同算法的时间复杂度、空间复杂度、稳定性等特性，并能根据问题的具体要求（数据规模、数据特性、内存限制等）做出合理的选择和必要的优化，是提升编程能力和解决实际问题的关键。排序作为基础操作，其思想和技巧（如分治、比较、交换、堆、自定义比较）贯穿于许多更复杂问题的解法之中。不断实践、分析和总结，才能更好地掌握这些强大的工具。